

소담 오브레인 (SoDam O-Brain) — 완전 가이드

컴퓨터·AI·IT를 처음 접하는 분도 이 가이드만 따라하면 됩니다. 어려운 용어에는 반드시 쉬운 설명을 붙였습니다. 이 문서와 GUIDE.pdf의 내용은 동일합니다.

목차

- O-Brain이란?
- 사전 준비물 · 필요 프로그램
- 다운로드 방법
- 설치 방법 (단계별)
- 빠른 시작
- 실행 방법
- 사용 방법 — 웹 대시보드
- 사용 방법 — MCP 도구 (Claude Code 연동)
- 작동 방법 (내부 구조)
- 명령어 전체 목록
- 워크플로우 (일반적인 하루 사용 패턴)
- 보안 · 데이터 흐름
- 아키텍처 (기술 구조)
- 파일 · 문서 위치
- 문제 · 오류 대처 방법
- FAQ (자주 묻는 질문)
- 법률 · 저작권 · 라이선스 · 상업적 용도

1. O-Brain이란?

O-Brain은 AI(Claude Code 등)와 나눈 대화에서 중요한 내용을 자동으로 추출·저장하고, 언제든지 꺼내 쓸 수 있게 해주는 완전 로컬 기억 시스템입니다.

쉬운 비유

AI는 매우 똑똑하지만, 대화가 끝나면 모든 것을 잊어버립니다. 마치 기억을 잃는 천재 친구와 대화하는 것과 같습니다.

O-Brain은 그 친구 옆에 두는 자동 메모 노트입니다. "포트는 7740으로 하자", "API 키는 절대 코드에 넣지 말자" 같은 말을 알아서 적어두고, 다음에 다시 알려줍니다.

핵심 특징

기능	설명
100% 로컬	모든 기억이 내 PC(SQLite 데이터베이스)에만 저장됩니다. 외부 서버·클라우드에 아무것도 전송되지 않습니다.
하이브리드 검색	키워드 검색(FTS5)과 의미 유사도 검색(벡터 검색)을 동시에 사용합니다. "비슷한 내용"도 찾아냅니다.
지식 그래프	기억들 사이의 연결을 2D/3D 그래프(점과 선)로 시각화합니다.
시간여행	날짜를 선택해 그 날 기준 "살아있던 기억"만 볼 수 있습니다.

신뢰도 감쇠	AI가 추출한 기억은 시간이 지날수록 신뢰도가 자동으로 낮아집니다 (30일 반감기).
보안 필터	저장 전 API 키·비밀번호·토큰을 자동으로 제거합니다(REDACTED 처리).
자동 백업	서버 시작 시·일괄 삭제 전 자동으로 스냅샷을 저장합니다.
MCP 연동	Claude Code에서 7개의 도구로 기억을 저장·검색·관계 관리합니다.
고아 노드 시각화	사용자 정의 관계가 없는 기억에 점선 테두리를 표시합니다.

2. 사전 준비물 · 필요 프로그램

반드시 필요한 것

항목	요구 사항	설명
운영체제	Windows 10 / 11 (64-bit)	맥(macOS)·리눅스(Linux)는 별도 테스트 필요
Node.js	20.x LTS 이상	프로그램을 실행하는 "엔진". nodejs.org에서 무료 다운로드
npm	Node.js에 포함됨	Node.js 설치 시 자동으로 함께 설치됩니다
저장 공간	500 MB 이상	AI 임베딩 모델(약 90 MB) + 데이터 저장 공간
메모리(RAM)	4 GB 이상 권장	8 GB 이상이면 더 쾌적합니다
웹 브라우저	Chrome, Edge, Firefox 등	대시보드 화면을 보기 위해 필요합니다

선택 사항 (MCP 연동 시 필요)

항목	설명
Claude Code	Anthropic이 만든 AI 코딩 도구. MCP 도구 연동 시 필요.
인터넷 연결	최초 설치 시와 임베딩 모델 첫 다운로드 시에만 필요. 이후 완전 오프라인 동작.

필요하지 않은 것

- API 키 불필요 (기본 동작에)
- 유료 구독 불필요
- 계정 가입 불필요
- 클라우드 서비스 불필요

3. 다운로드 방법

이미 파일이 있는 경우

이 가이드를 읽고 있다면 O-Brain 파일이 이미 내 컴퓨터에 있습니다. 프로젝트 폴더 위치를 확인하세요:

```
D:\WAI_Dev_Work\2026y\26y_06m_21d_SoDam_0-BrainW
```

또는 파일을 받은 폴더를 확인하면 됩니다.

Git으로 받는 경우 (개발자용)

```
git clone [저장소 주소]
cd [폴더명]
```

저장소가 비공개(private)일 경우 접근 권한이 필요합니다.

4. 설치 방법 (단계별)

4-1단계: Node.js 설치 (처음 한 번만)

Node.js는 JavaScript(자바스크립트) 프로그램을 실행하는 "엔진"입니다. O-Brain은 JavaScript로 만들어졌기 때문에 이 엔진이 반드시 필요합니다.

1. 웹 브라우저에서 **nodejs.org** 접속
2. 초록색 "**LTS**" 버튼 클릭 (LTS = Long-Term Support, 안정 버전)
3. 설치 파일 다운로드 후 실행
4. 설치 화면에서 "Next" → "Next" → "Install" → "Finish" (기본값으로 진행)
5. 설치 완료 후 **터미널(명령 프롬프트)을 새로 열기**

터미널 여는 방법: Windows 키 + R → `cmd` 입력 → Enter 또는 Windows 키 → "PowerShell" 검색 → 클릭

6. 아래 명령어 입력해서 버전 확인:

```
node --version
```

v20.x.x 또는 그 이상이 표시되면 정상입니다.

```
npm --version
```

숫자(예: 10.x.x)가 표시되면 정상입니다.

4-2단계: 의존성 설치

"의존성(Dependency)"이란 O-Brain이 작동하기 위해 필요한 외부 도구 모음입니다. 인터넷에서 자동으로 다운로드 됩니다.

1. 터미널에서 `app` 폴더로 이동:

```
cd D:\WAI_Dev_Work\2026y\26y_06m_21d_SoDam_O-BrainWapp
```

2. 의존성 설치 명령어 실행:

```
npm install
```

3. 완료까지 1~5분 정도 기다립니다.

4. 설치 완료 후 오류 메시지 없이 프롬프트가 돌아오면 정상입니다.

5. 빠른 시작

설치가 완료되었다면 아래 3단계로 바로 시작할 수 있습니다.

1. 터미널에서 서버 시작:

```
cd D:\WAI_Dev_Work\W2026y\W26y_06m_21d_SoDam_0-BrainWapp
npm start
```

2. 첫 실행 시 안내:

```
O-Brain 로컬 서버 ▶ http://127.0.0.1:7740 (임베딩: local)
[backup] 스냅샷 저장: ...
```

처음 실행할 때: AI 임베딩 모델(약 90 MB)을 자동으로 다운로드합니다. 최초 1회만 1~3분이 더 걸립니다. 이후 실행부터는 바로 시작됩니다.

3. 브라우저에서 대시보드 열기:

브라우저 주소창에 아래 주소를 입력하고 Enter:

```
http://127.0.0.1:7740
```

O-Brain 대시보드(화면)가 열리면 성공입니다.

6. 실행 방법

기본 실행

```
# app 폴더에서 실행
cd D:\WAI_Dev_Work\W2026y\W26y_06m_21d_SoDam_0-BrainWapp
npm start
```

포트 변경 (기본 포트 7740이 사용 중일 때)

```
# Windows PowerShell
$env:OBRAIN_PORT=7741; npm start

# 또는 .env.local 파일에 추가:
# OBRAIN_PORT=7741
```

서버 종료

터미널에서 `Ctrl + C` 를 누릅니다.

자동 시작 설정 (선택사항)

Windows 시작 시 자동으로 실행하려면 Windows 작업 스케줄러를 사용할 수 있습니다. (방법: Windows 키 → "작업 스케줄러" 검색)

7. 사용 방법 — 웹 대시보드

브라우저에서 `http://127.0.0.1:7740` 을 열면 대시보드가 표시됩니다.

7-1. 화면 구성 (상단 탭)

탭	설명
그래프	기억들을 점과 선으로 표시하는 지식 맵. 첫 화면.
개요	전체 현황: 총 기억 수, 유형별/카테고리별 분포, 연결 많은 기억
목록	기억을 카드 형태로 나열. 스크롤로 탐색.
타임라인	시간순으로 기억 표시

7-2. 그래프 탭 사용법

그래프란? 기억들이 점(노드)으로 표시되고, 비슷하거나 연관된 기억끼리 선(엣지)으로 연결됩니다.

- **마우스를 점 위에 올리면:** 연결된 기억들이 강조됩니다
- **점을 클릭하면:** 오른쪽에 상세 정보가 열립니다
- **검색창:** 위쪽 검색창에서 검색하면 해당 점이 밝아집니다
- **2D/3D 전환:** 상단의 "2D"/"3D" 버튼으로 전환
- **확대/축소:** 마우스 휠 또는 우하단 버튼
- **점선 테두리:** 사용자 정의 관계가 없는 "고아 기억"에 표시됩니다

색상 의미:

- 점 크기 = 중요도 + 연결 수 (클수록 중요)
- "분류색" 버튼: 유형별/카테고리별로 색상 변경

7-3. 기억 상세 보기

목록이나 그래프에서 기억을 클릭하면 오른쪽에 상세 패널이 열립니다:

- **내용:** 저장된 기억 텍스트
- **유형:** 지식/결정/약속/금지 등
- **중요도:** ★ 별점 (1~5)
- **신뢰도:** 0.1~1.0 (AI 추출 기억은 시간이 지나면 자동으로 낮아짐)
- **카테고리:** 자동 분류된 주제
- **생성일:** 기억이 저장된 날짜
- **조회수:** 이 기억을 열어본 횟수 (많이 볼수록 그래프에서 더 빛남)

7-4. 기억 편집 및 삭제

상세 패널에서:

- **편집(✎):** 내용, 유형, 중요도 수정 가능. 편집 시 비밀 정보 자동 제거 재실행.
- **삭제:** 해당 기억을 영구 삭제. 삭제 전 자동 백업 권장.
- **복사:** 내용을 클립보드에 복사

7-5. 기억 추가 (UI에서 직접)

상단 "+" 버튼 또는 API로 기억을 직접 추가할 수 있습니다:

- 내용 입력
- 유형 선택 (지식/결정/약속 등)
- 중요도 선택 (1~5)
- 저장 클릭 → 자동으로 의미 분석 및 임베딩 생성

7-6. 기억 관계 연결

기억 상세 패널 하단 "관계" 섹션에서:

1. "관계 추가" 클릭
2. 연결할 다른 기억 ID 입력
3. 관계 유형 선택: SUPERSEDES(번복), SUPPORTS(뒷받침), INFLUENCES(영향), CONTRADICTS(충돌)
4. 저장

SUPERSEDES 선택 시: 자동으로 대상 기억의 유효 종료일(`valid_until`)이 오늘로 설정됩니다.

7-7. 검색

상단 검색창에서:

- 하이브리드 검색: 키워드 + 의미 유사도를 동시에 검색
- 예시: "포트 설정" 검색 → "포트"가 포함된 기억과 비슷한 내용의 기억 모두 표시

7-8. 필터 · 정렬

상단 필터바에서:

- 범위(scope): 전체(global) / 프로젝트(project) 구분
- 정렬: 최신순 / 중요도순
- 기간 필터: 날짜 범위 지정

7-9. 시간여행 기능 (Phase 2)

날짜 입력창에서 과거 날짜를 선택하면:

- 그 날짜 기준으로 "살아있던 기억"만 표시됩니다
- SUPERSEDES로 무효화된 기억은 무효화 날짜 이후에는 표시되지 않습니다
- 기억 변천 과정을 추적할 때 유용합니다

예시: 2026-06-01을 선택하면 그 날짜까지 유효했던 기억만 볼 수 있습니다.

7-10. 내보내기

상단 우측 "내보내기" 버튼:

- JSON 형식: 프로그램이 읽을 수 있는 데이터 형식
- 마크다운 형식: 텍스트 편집기에서 읽기 좋은 형식

7-11. 일괄 삭제

여러 기억을 한 번에 선택해서 삭제할 수 있습니다.

주의: 일괄 삭제 전에 자동으로 백업이 생성됩니다. 삭제 후 복구가 필요하면 `app/data/backups/` 폴더의 최신 백업 파일을 사용하세요.

8. 사용 방법 — MCP 도구 (Claude Code 연동)

****MCP(Model Context Protocol)****는 Claude Code가 외부 도구를 사용할 수 있게 해주는 규격입니다. O-Brain의 MCP 서버를 등록하면 Claude Code 대화 중에 기억을 저장·검색할 수 있습니다.

8-1. MCP 서버 등록

Claude Code의 설정 파일(`~/.claude/settings.json`)에 아래 내용 추가:

```
{
  "mcpServers": {
    "o-brain": {
      "command": "node",
      "args": [ "C:/절대경로/app/src/mcp-server.mjs" ]
    }
  }
}
```

경로에 백슬래시(`\`) 대신 슬래시(`/`)를 사용하세요.

등록 후 Claude Code를 재시작합니다.

8-2. MCP 도구 목록

도구 이름	하는 일	사용 예
save_memory	기억 저장	"이 결정을 기억해줘"
search_memory	키워드/의미로 검색	"포트 설정 관련 기억 찾아줘"
get_memory	특정 ID의 기억 조회	기억 ID로 상세 정보 확인
get_related	연관 기억 목록	이 기억과 관련된 것들
get_timeline	시간순 기억 목록	최근 기억들 보기
add_relation	두 기억 사이 관계 추가	"A가 B를 반복했다고 연결해줘"
list_categories	카테고리별 기억 수	어떤 주제가 많은지 확인

8-3. Claude Code에서 사용 예시

나: "방금 포트 7740으로 결정했어. 기억해줘."
Claude: (save_memory 도구 실행) "포트 7740 결정을 기억에 저장했습니다."

나: "이전에 포트 관련 결정 뭐 했었지?"
Claude: (search_memory 도구 실행) "저장된 기억 중 포트 관련: ..."

9. 작동 방법 (내부 구조)

9-1. 기억 저장 과정

- 입력: MCP 도구나 웹 UI에서 텍스트 입력

- 2. 보안 필터: API 키·비밀번호 등 민감 정보 자동 제거
- 3. 중복 확인: 이미 있는 기억과 비교해 중복이면 건너뛰
- 4. 임베딩: 텍스트를 384차원 숫자 벡터로 변환 (all-MiniLM-L6-v2 모델, 로컬 실행)
- 5. 저장: SQLite 데이터베이스에 저장 (텍스트 + 벡터 + 메타데이터)
- 6. 카테고리 분류: 규칙 기반으로 주제 자동 분류

9-2. 검색 과정 (하이브리드 검색)

- 1. 키워드 검색(FTS5): 정확한 단어 매칭
- 2. 벡터 검색: 의미적으로 유사한 내용 검색
- 3. RRF 병합: 두 검색 결과를 합쳐 최종 순위 결정 (C=60 상수 사용)
- 4. 결과 반환: 상위 결과를 관련도 순으로 반환

9-3. 신뢰도 감쇠 (Confidence Decay)

AI가 추출한 기억은 시간이 지날수록 신뢰도가 자동으로 낮아집니다:

- 공식: 새 신뢰도 = 이전 신뢰도 × 0.5^(경과일수/30)
- 의미: 30일이 지나면 신뢰도가 절반으로, 60일이면 1/4로 낮아짐
- 예외: 사용자가 직접 입력한 기억(source='user')과 신뢰도 0.99 이상은 감쇠 안 됨
- 최솟값: 0.1 미만으로는 내려가지 않음
- 추가: 서버 시작 시 마지막 적용 후 12시간 이상 지난 경우 실행

9-4. 그래프 구성

- 노드: 각 기억이 하나의 점
- 엣지(자동): 임베딩이 비슷한 기억끼리 자동으로 연결 (sqlite-vec KNN)
- 엣지(수동): 사용자가 직접 추가한 관계 (SUPERSEDES/SUPPORTS 등) — 색깔 선으로 표시
- 크기: 중요도 + 연결 수에 비례
- 성능 제한: 최대 2000개 노드만 표시 (중요도 높은 것 우선)

9-5. 로컬 API 토큰

서버가 시작될 때마다 새로운 임의 토큰을 생성합니다:

- 브라우저에 자동으로 주입되어 API 요청 시 함께 전송
- data/.api-token 파일에도 저장 (MCP 서버용)
- 서버 재시작 시 새 토큰 발급 (보안 강화)

10. 명령어 전체 목록

10-1. npm 스크립트

app/ 폴더에서 실행:

명령어	하는 일
npm start	로컬 서버 시작 (기본 포트 7740)
npm run selftest	자가 진단 (저장·검색·보안 테스트)
npm run backup	수동 백업 생성
npm run status	DB 상태 및 통계 조회
npm run seed	예제 데이터 입력 (처음 테스트용)

npm run search	검색 데모 (터미널용)
----------------	--------------

10-2. MCP 도구 (Claude Code에서 사용)

도구	입력	출력
save_memory	content, type, importance, source, scope	저장된 기억 ID
search_memory	query, limit	검색 결과 목록
get_memory	id	기억 상세 정보
get_related	id, limit	연관 기억 목록
get_timeline	limit, offset	시간순 기억 목록
add_relation	from_id, to_id, type	관계 생성 결과
list_categories	—	카테고리별 기억 수

10-3. HTTP API 엔드포인트

서버가 실행 중일 때 `http://127.0.0.1:7740` 에서 접근 가능:

메서드	경로	설명
GET	/api/health	서버 상태 확인
GET	/api/memories	기억 목록 (페이지네이션)
POST	/api/memory	기억 추가
GET	/api/memory/:id	기억 단건 조회
PATCH	/api/memory/:id	기억 수정
DELETE	/api/memory/:id	기억 삭제
POST	/api/memory/batch-delete	일괄 삭제
GET	/api/search	검색
GET	/api/graph	그래프 데이터
GET	/api/stats	통계
GET	/api/categories	카테고리 목록
GET	/api/relations	관계 목록
POST	/api/relation	관계 추가
DELETE	/api/relation/:id	관계 삭제

10-4. 환경 변수 (`.env.local`)

`app/` 폴더에 `.env.local` 파일을 만들어 설정 가능:

변수	기본값	설명
OBRAIN_PORT	7740	서버 포트 번호
OBRAIN_DATA_DIR	./data	데이터 저장 폴더 경로

11. 워크플로우 (일반적인 하루 사용 패턴)

일반 사용자 (웹 UI 중심)

아침: `npm start` 로 서버 시작
→ `http://127.0.0.1:7740` 열기
→ 어제 저장된 기억 확인

작업 중: 웹 UI에서 새 기억 직접 입력
중요한 결정이 생기면 바로 기록

저녁: 오늘 추가한 기억 확인
관련 기억끼리 관계 연결 (SUPPORTS, INFLUENCES 등)
필요 없는 기억 정리 · 삭제

주간: `npm run backup` 으로 수동 백업 확인
오래된 기억들 신뢰도 상태 확인

Claude Code 사용자 (MCP 연동)

대화 시작: Claude Code가 관련 기억을 자동으로 참조
"포트 설정이 뭐였지?" → `search_memory`로 즉시 찾기

대화 중: 중요한 결정 → `save_memory`로 즉시 저장
"이 결정을 기억해줘" → 자동 처리

대화 후: 웹 UI에서 저장된 기억 시각화 확인

기억 정리 워크플로우

- 웹 UI의 **개요** 탭에서 전체 현황 파악
- 그래프** 탭에서 점선 테두리(고아 노드) 기억 확인
- 관계가 없는 중요 기억들 관계 연결
- 목록** 탭에서 오래됐거나 불필요한 기억 삭제
- 시간여행** 기능으로 과거 기억 상태 점검

12. 보안 · 데이터 흐름

12-1. 핵심 보안 원칙

O-Brain은 "**Secure by Design**" 원칙을 따릅니다: 기능보다 먼저 안전, 기본 설정이 가장 안전한 상태.

12-2. 네트워크 보안

항목	설명
바인딩 주소	127.0.0.1 (내 PC 전용). 0.0.0.0(모든 기기)으로 절대 열리지 않음
CORS 정책	localhost 또는 127.0.0.1에서 온 요청만 허용. 다른 사이트 요청 거부
HTTPS	로컬 전용이므로 HTTP 사용. 외부 노출 없음

12-3. 보안 헤더

서버가 자동으로 적용하는 HTTP 보안 헤더:

헤더	값	의미
Content-Security-Policy	default-src 'self'	같은 출처 리소스만 허용
X-Content-Type-Options	nosniff	콘텐츠 타입 위변조 방지
X-Frame-Options	DENY	iframe 삽입 차단
Referrer-Policy	no-referrer	참조 URL 전송 안 함

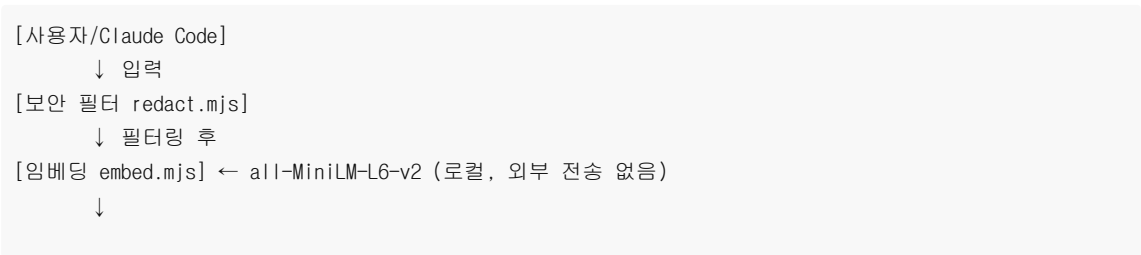
12-4. 데이터 보안

항목	처리 방법
API 키 감지	sk-, AKIA, Bearer 등 패턴 → [REDACTED]로 자동 치환
비밀번호	password=, secret= 등 → 자동 제거
저장 전 검사	모든 content 필드는 redact.mjs를 통해 저장
편집 시 재검사	내용 수정 시에도 자동 재필터링

12-5. 파일 시스템 보안

항목	설명
data/ 폴더	.gitignore에 포함 → Git에 절대 올라가지 않음
.env.local	.gitignore에 포함 → 비밀 설정 파일 보호
*.sqlite	.gitignore에 포함 → 데이터베이스 파일 보호
API 토큰 파일	data/.api-token, 권한 0600 (소유자만 읽기)

12-6. 데이터 흐름 다이어그램



```

[SQLite DB]
├── memory 테이블 (기억 본문 + 메타데이터)
├── memory_fts (FTS5 전문 검색 인덱스)
├── memory_vec (벡터 검색 인덱스, 384차원)
├── relation 테이블 (기억 간 관계)
├── session 테이블 (대화 세션 기록)
└── app_settings 테이블 (서버 설정 저장)
    ↓
[search.mjs] → FTS5 + 벡터 하이브리드 검색
    ↓
[server.mjs] → HTTP API (127.0.0.1:7740 전용)
    ↓
[브라우저/MCP 클라이언트]

```

12-7. 로컬 API 토큰 흐름

```

서버 시작 → randomBytes(16) 생성 → API_TOKEN
    ↓
index.html에 주입 (브라우저 자동 전송)
data/.api-token에 저장 (MCP 서버가 읽어서 사용)

```

12-8. 클라우드 동기화 주의

data/ 폴더를 Dropbox, OneDrive, Google Drive 등 클라우드 동기화 폴더에 두지 마세요:

- 기억 데이터가 클라우드에 올라갈 수 있습니다
- SQLite 파일 동기화 충돌이 발생할 수 있습니다

13. 아키텍처 (기술 구조)

13-1. 전체 구조

```

0-Brain
├── 백엔드 (서버)
│   ├── Node.js + Express.js v5
│   │   ├── PORT: 7740
│   │   ├── HOST: 127.0.0.1 (외부 차단)
│   │   └── ES Modules (type: "module")
│   └── 데이터베이스
│       ├── SQLite (better-sqlite3)
│       │   ├── FTS5 (전문 검색)
│       │   └── sqlite-vec (벡터 검색, 384차원)
│       └── AI 임베딩
│           ├── @huggingface/transformers
│           └── all-MiniLM-L6-v2 (완전 로컬, 다운로드 1회)
├── 그래프 시각화
│   ├── force-graph (2D)
│   └── 3d-force-graph (3D)
│       ※ 두 라이브러리 모두 /vendor/ 로컬 제공 (CDN 불사용)
└── MCP 서버

```

└── @modelcontextprotocol/sdk
└── 7개 도구 제공

13-2. 소스 파일 역할

파일	역할
src/server.mjs	HTTP API 서버. 모든 API 엔드포인트 처리
src/db.mjs	SQLite 데이터베이스 초기화 및 마이그레이션
src/store.mjs	기억 CRUD·검색·관계 관리·신뢰도 감쇠
src/embed.mjs	텍스트 → 벡터 변환 (임베딩)
src/search.mjs	하이브리드 검색 (FTS5 + 벡터 + RRF)
src/graph.mjs	그래프 데이터 생성 (노드·엣지·고아 판별)
src/redact.mjs	민감 정보 필터링
src/backup.mjs	자동/수동 백업
src/mcp-server.mjs	MCP 서버 (Claude Code 연동)
src/classify.mjs	기억 카테고리 자동 분류
web/index.html	웹 대시보드 UI (바닐라 JS)

13-3. 데이터베이스 스키마

memory 테이블 (핵심):

컬럼	타입	설명
id	INTEGER PK	기억 고유 번호
content	TEXT	기억 내용 (비밀정보 제거됨)
type	TEXT	유형 (지식/결정/약속/금지 등)
importance	INTEGER	중요도 (1~5)
confidence	REAL	신뢰도 (0.1~1.0)
source	TEXT	출처 (ai/user/mcp)
project	TEXT	프로젝트 이름
scope	TEXT	범위 (global/project)
category	TEXT	자동 분류 카테고리
valid_from	TEXT	유효 시작일
valid_until	TEXT	유효 종료일 (SUPERSEDES 시 자동 설정)
invalidated_by	INTEGER	무효화한 기억 ID

access_count	INTEGER	조회 횟수
created_at	TEXT	생성 시각

relation 테이블:

컬럼	타입	설명
id	INTEGER PK	관계 고유 번호
from_id	INTEGER	시작 기억 ID
to_id	INTEGER	대상 기억 ID
type	TEXT	관계 유형 (SUPERSEDES 등)
created_at	TEXT	생성 시각

14. 파일 · 문서 위치

14-1. 전체 폴더 구조

프로젝트 루트/	
├── .PRD/	제품 요구사항 문서 12개 (개발자용)
├── app/	로컬 서버 앱 (핵심)
├── src/	소스 코드
├── server.mjs	
├── db.mjs	
├── store.mjs	
├── embed.mjs	
├── search.mjs	
├── graph.mjs	
├── redact.mjs	
├── backup.mjs	
├── mcp-server.mjs	
└── classify.mjs	
├── web/	
└── index.html	웹 대시보드 UI
├── data/	← 개인 데이터 (Git에 올라가지 않음)
├── obrain.db	SQLite 데이터베이스 (기억 저장소)
├── backups/	자동 백업 파일들
└── .api-token	로컬 API 토큰
├── node_modules/	
├── package.json	
└── .env.local	
├── README.md	한국어 간략 안내
└── README.en.md	영문 간략 안내

— GUIDE.md	한국어 완전 가이드 (이 파일)
— GUIDE.en.md	영문 완전 가이드
— LICENSE	Apache License 2.0 전문

14-2. 중요 파일 위치 요약

파일	역할
app/data/obrain.db	가장 중요: 모든 기억이 저장된 데이터베이스
app/data/backups/	자동 백업 파일들 (최신 10개 보관)
app/data/.api-token	서버 세션 토큰 (재시작마다 갱신)
app/.env.local	포트 등 개인 설정 (없어도 기본값으로 동작)

14-3. 데이터 백업 및 복원

자동 백업: 서버 시작 시 일괄 삭제 전 자동 실행

수동 백업:

```
cd app
npm run backup
```

복원 방법:

- app/data/backups/ 폴더에서 원하는 날짜의 .db 파일 확인
- app/data/obrain.db 를 다른 곳에 복사 (현재 버전 보존)
- 백업 파일을 app/data/obrain.db 로 복사

15. 문제 · 오류 대처 방법

15-1. 설치 관련

증상	원인	해결 방법
'node'은(는) 내부 또는 외부 명령이 아닙니다	Node.js 미설치 또는 새 창 안 열음	nodejs.org에서 LTS 설치 후 터미널 새로 열기
npm install 중 오류	네트워크 문제 또는 Node.js 버전	인터넷 연결 확인. node --version이 v20+ 인지 확인
better-sqlite3 빌드 실패	Node.js 버전 불일치	npm install --force 또는 Node.js v20 LTS 재설치
ERR_MODULE_NOT_FOUND	의존성 미설치	npm install 실행

15-2. 서버 실행 관련

증상	원인	해결 방법
----	----	-------

EADDRINUSE: address already in use :::7740	포트 7740 이미 사용 중	\$env:OBRAIN_PORT=7741; npm start (다른 포트 사용)
서버가 시작되지 않음	여러 원인	오류 메시지 전체를 확인. npm run selftest 실행
서버 시작 후 브라우저 접속 안 됨	방화벽 문제	Windows 방화벽에서 Node.js 허용. 주소가 http://127.0.0.1:7740 인지 확인

15-3. 대시보드 화면 관련

증상	원인	해결 방법
그래프가 비어 있음	기억이 없음	npm run seed로 예제 데이터 추가하거나 직접 기억 추가
3D 그래프가 안 보임	WebGL(그래픽 가속) 미지원	2D 모드로 전환. 또는 Chrome/Edge 최신 버전 사용
새 기억이 화면에 안 보임	화면 갱신 필요	우상단 새로고침(🔄) 버튼 클릭
화면이 흰 화면	서버 미실행	npm start 실행 후 새로고침

15-4. 검색 관련

증상	원인	해결 방법
검색 결과가 없음	기억 자체가 없거나 임베딩 미생성	기억 추가 후 재검색. npm run selftest 실행
관련 없는 결과가 나옴	검색어가 너무 짧거나 일반적	더 구체적인 단어 사용
검색이 느림	임베딩 모델 초기화 중	첫 실행 후 잠시 기다렸다가 검색

15-5. 임베딩 모델 관련

증상	원인	해결 방법
첫 실행이 너무 느림	임베딩 모델 다운로드 중 (~90MB)	1~3분 기다리기. 이후 실행은 빠름
임베딩 모델 다운로드 실패	인터넷 차단	인터넷 연결 확인. 방화벽에서 Node.js 허용

15-6. MCP 연동 관련

증상	원인	해결 방법
Claude Code에서 MCP 도구 안 보임	settings.json 설정 오류	경로가 절대 경로 인지 확인. 슬래시(/) 사용
MCP 도구 호출 실패	서버 미실행	npm start로 서버 먼저 실행
경로 오류	백슬래시 사용	C:\W... → C:/... 로 변경

15-7. 데이터 관련

증상	원인	해결 방법
기억을 실수로 삭제	삭제는 즉시 적용	app/data/backups/ 에서 최신 파일 복원
DB 파일 손상	비정상 종료	백업에서 복원. 이후 정상 종료(Ctrl+C) 사용
기억에 API 키가 그대로 저장됨	비표준 형식이라 필터가 놓침	해당 기억 즉시 삭제 또는 편집 → 해당 키 즉시 재발급

16. FAQ (자주 묻는 질문)

Q. 데이터는 어디에 저장되나요?

A. 모든 기억은 `app/data/obrain.db` (SQLite 파일)에 저장됩니다. 이 파일은 내 컴퓨터 안에만 있으며, 외부로 전송되지 않습니다.

Q. 인터넷이 필요한가요?

A. 최초 설치 시(`npm install`)와 임베딩 모델 첫 다운로드 시에만 필요합니다. 이후에는 완전히 오프라인으로 동작합니다.

Q. Claude Code 없이 사용할 수 있나요?

A. 네. 웹 대시보드(`http://127.0.0.1:7740`)만으로도 기억 추가·검색·관계 관리를 모두 할 수 있습니다. Claude Code는 MCP 도구 연동 시에만 필요합니다.

Q. 비용이 드나요?

A. O-Brain 자체는 무료입니다. Claude Code나 Anthropic API를 사용하는 경우 해당 서비스의 요금이 적용될 수 있습니다.

Q. 기억이 실수로 삭제됐어요. 복구할 수 있나요?

A. 서버 시작 시와 일괄 삭제 전 자동 백업이 생성됩니다. `app/data/backups/` 폴더의 최신 `.db` 파일로 복원할 수 있습니다. (단, 단건 삭제 전 백업은 자동 생성되지 않으므로 중요 기억은 정기적으로 `npm run backup` 실행 권장)

Q. 포트를 변경할 수 있나요?

A. 네. `app/.env.local` 파일에 `OBRAIN_PORT=7741` 추가 후 서버 재시작하면 됩니다.

Q. 여러 프로젝트를 구분해서 관리할 수 있나요?

A. 기억 저장 시 `scope=project` 와 `project` 이름을 지정하면 프로젝트별로 분류됩니다. 검색 및 목록 조회 시 `scope` 필터로 특정 프로젝트 기억만 볼 수 있습니다.

Q. Mac이나 Linux에서도 동작하나요?

A. Windows 기준으로 개발되었습니다. macOS-Linux에서는 기본적으로 동작하지만 `better-sqlite3` 네이티브 모듈 빌드에서 오류가 날 수 있습니다. 오류 시 `npm install --force` 또는 Node.js LTS 재설치를 시도해 보세요.

Q. 신뢰도가 계속 낮아지는데 어떻게 해야 하나요?

A. 신뢰도 감쇠는 AI가 추출한 기억에만 적용됩니다. 직접 입력(`source=user`)하거나 신뢰도를 0.99 이상으로 설정하면 감쇠가 적용되지 않습니다.

Q. "고아 기억(점선 테두리)"이란 무엇인가요?

A. 사용자가 직접 추가한 관계(SUPERSEDES, SUPPORTS 등)가 없는 기억입니다. 점선 테두리 기억들을 확인하고 관계를 추가하면 지식 그래프가 더 풍부해집니다.

Q. 시간여행 기능은 어떻게 사용하나요?

A. 웹 대시보드 상단의 날짜 입력 필드에서 날짜를 선택하면, 그 날짜 기준 "살아있던" 기억만 표시됩니다. 날짜를 비우면 현재 상태(기본값)로 돌아옵니다.

17. 법률 · 저작권 · 라이선스 · 상업적 용도

 **중요 안내:** 이 섹션은 일반 정보 제공 목적입니다. 법적 효력이 있는 조언이 아니며, 구체적인 법률 문제는 반드시 전문가에게 확인하세요.

17-1. 라이선스

Apache License 2.0 © SoDam AI Studio, 2026

이 소프트웨어는 **Apache License 2.0**에 따라 배포됩니다. 라이선스 전문은 프로젝트 루트의 `LICENSE` 파일에 있습니다.

17-2. 저작권자

저작권자: SoDam AI Studio 연도: 2026

17-3. 허용 사항 (Apache-2.0 기준)

허용 항목	설명
개인 사용	개인 목적으로 자유롭게 사용
수정	소스 코드를 수정해서 사용
복제	복사본 제작
재배포	수정 여부와 관계없이 재배포
상업적 사용	상업 목적으로 사용·판매·납품
특허 사용	기여자의 특허 사용 허가 포함

17-4. 의무 사항 (반드시 지켜야 할 것)

재배포 또는 상업적 사용 시 반드시 지켜야 할 사항:

- 저작권 고지 보존:** © SoDam AI Studio, 2026 문구를 유지해야 합니다
- 라이선스 고지 보존:** Apache-2.0 라이선스 고지를 포함해야 합니다
- 수정 사항 명시:** 원본을 수정한 경우 수정 사실을 명시해야 합니다
- NOTICE 파일 포함:** 배포 시 `NOTICE` 파일(있는 경우)을 포함해야 합니다

17-5. 보증 없음 (AS IS)

이 소프트웨어는 어떠한 보증도 제공하지 않습니다.

사용에 따른 모든 결과와 리스크는 전적으로 사용자 책임입니다.

17-6. 외부 서비스 약관 별도 적용

서비스	약관	O-Brain 라이선스와 관계
Anthropic/Claude	Anthropic 이용약관	독립적. O-Brain 라이선스와 무관
Claude Code	Anthropic 이용약관	독립적. O-Brain 라이선스와 무관

17-7. 의존성 라이선스

라이브러리	추정 라이선스	비고
better-sqlite3	MIT	공식 확인 권장
sqlite-vec	Apache-2.0/MIT	공식 확인 권장
@huggingface/transformers	Apache-2.0	공식 확인 권장
all-MiniLM-L6-v2 (AI 모델)	Apache-2.0	모델 카드 직접 확인 권장
express	MIT	공식 확인 권장
force-graph	MIT	공식 확인 권장
3d-force-graph	MIT	공식 확인 권장
@modelcontextprotocol/sdk	MIT	공식 확인 권장

17-8. 상표 사용 주의

- "Claude", "Anthropic" 등은 해당 회사의 상표입니다
- O-Brain은 이들과 제휴 관계가 없으며, 공식 인증을 받지 않았습니다

17-9. 개인정보 및 데이터 보호

- O-Brain에 저장하는 기억(데이터)은 전적으로 사용자의 책임하에 관리됩니다
- 개인정보(이름·주소·계좌 등)를 기억에 저장하지 않도록 주의하세요
- O-Brain 개발자(SoDam AI Studio)는 사용자 데이터에 접근할 수 없으며, 접근하지 않습니다

이 가이드의 모든 내용은 2026년 6월 기준 실제 동작 검증된 내용을 바탕으로 작성되었습니다.

영문 버전: [GUIDE.en.md](#) · 간략 안내: [README.md](#) (한국어) · [README.en.md](#) (영문)